Exp. No.: Date. Regd.No. 7 R Page No.

Two pointers & sliding window:

- Form 0 : Fixed size window
- Form 1 : Variable size window
- Form 2 : Two sum & and minimum the distance by maximizing answer.
- From 3 : Multi sequence.

Form 0: Fixed size window → contigoues away.

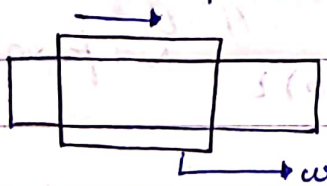
1. knowing about subaway?
2. code to find all subaways? → $O(n^3)$, $O(n^2)$ → Analyizing what could be task repeatetted

And knowing about what we really asked in this form?

→ Asked to maintain a fixed size subaway of length k,

→ figuring out importance of the maintaining window of fixed size.
and how window would be titted and according to that
How we maintain the sum.

- understanding about prefix sum importance
- understanding importance of running sum.



for(0 → k-1){

}

Q: [3, -1, -1, 0, 1]

k=3

figure out if subaway with sum = 0 exists.

for(tail=0, head=k; head<n; tail++, head++){

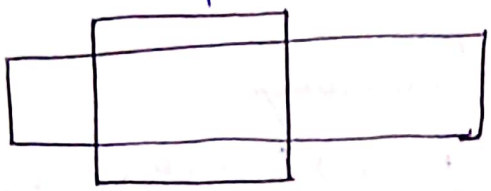
}

Scanned with OKEN Scanner

Form-1: Vaiable size window:

→ Here window size is not fixed and something keep varies according to the condition.

→ vaiable size window.



How things work over here in the perceptive of sudo code:

→ Same like snake game eating as much as we can

→ first window creation ~~&~~ ~~to~~ ~~so~~ starting we are increasing our window according to our feasibility.

sudo code:

according to our Requirement we put condition something is feasiable or Not feasiable.

```
tail = 0; head = -1;
while (tail < length)?
    while (head+1 < n && condition)?
        head++;
    // condition to insert value to d.s
    if (condition)?
    {
    }
    if (tail > head)?
        // their would no such window exist.
        tail++;
        head = tail-;
    } else? // Removing the value
        d.s remove(tail); from d.s bcz window has slided.
        tail++; }
```


Scanned with OKEN Scanner

Exp. No.: Date. Regd.No. 7 R Page No.

How the things were planned in variable size window:

1. Desiging data structure
2. Desiging check to handle data.
3. Applying template.

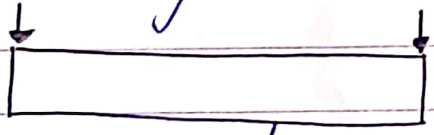
Form-2 Two Sum:

Secinario:

→ Something we are asked about the sum of two elements where it should be equal to the target.

and we were compromised order and they shouldn't be contigues. and when we listen we shouldn't follow order it gives us hint to changing order would be feasiable.

So we sort away.



→ and when we hit greater than target we try to shift the right pointer bcz by shifting right somehow we can decrease the sum ~~compared~~

→ and same applicable for left when we hit sum < target.

→ and sudo code:

```
while( st < end ){
    if( au[st] + au[end] < target ){
        stat++;
    }
    else if( au[st] + au[end] > target ){
        end--;
    }
    else{
        return stat, end };
```


Scanned with OKEN Scanner

→ and in the case of 3sum we select one element and we go for 2sum for remaining elements.

```
for(i=0; i<n; i++){
    if(a[i]==a[i-1]) continue;
    twopointer(0, end, target-a[i]);
}
```

Form-3: Multisequence: → Merging 2 sorted arrays.

self explanatory



How we design our approach

→ seeing constraints → $10^{\wedge}5$

→ we are not allowed to $O(n^2)$

and something we should think less than n^2

- → $n * \log(n)$
- → n
- → $\sqrt{n}$
- → 1

→ if they mention something like subarray we should think

- → kadan's algorithm → 1D format
- → sliding window → 2D format
  - → fixed size
  - → variable size
  - → 2sum
  - → Multisequence.

Scanned with OKEN Scanner